

```

with_items:
  - python2-docker
  - docker

- service:
    name: docker
    state: started
- name: Run the hello-world container
  docker_container:
    name: hello-world
    image: library/hello-world

```

The Parabola package repository is updated before proceeding to install the dependencies. The *python2-docker* package is required for use with Ansible. Hence, it is installed along with the *docker* package. The Docker daemon service is then started and the *library/hello-world* container is fetched and executed. A sample invocation and execution of the above playbook is shown below:

```

$ ansible-playbook playbooks/configuration/docker.yml -K
--tags=setup
SUDO password:

PLAY [Setup Docker] *****

TASK [Gathering Facts] *****
ok: [localhost]

TASK [Update the software package repository] *****
changed: [localhost]

TASK [Install dependencies] *****
ok: [localhost] => (item=python2-docker)
ok: [localhost] => (item=docker)

TASK [service] *****
ok: [localhost]

TASK [Run the hello-world container] *****
changed: [localhost]

PLAY RECAP *****
localhost                : ok=5    changed=2
unreachable=0    failed=0

```

With the verbose '-v' option to ansible-playbook, you will see an entry for LogPath, such as */var/lib/docker/containers/<container-id>/<container-id>-json.log*. In this log file, you will see the output of the execution of the *hello-world* container. This output is the same when you run the container manually as shown below:

```
$ sudo docker run hello-world
```

```
Hello from Docker!
```

This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the *hello-world* image from the Docker Hub.
 3. The Docker daemon created a new container from that image, which runs the executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.
- To try something more advanced, you can run an Ubuntu container with:

```
$ docker run -it ubuntu bash
```

You can share images, automate workflows, and more with a free Docker ID at <https://cloud.docker.com/>.

For more examples and ideas, do visit <https://docs.docker.com/engine/userguide/>.

An example

A deep learning (DL) Docker project is available (<https://github.com/floydhub/dl-docker>) with support for frameworks, libraries and software tools. We can use Ansible to build the entire DL container from the source code of the tools. The base OS of the container is Ubuntu 14.04, and will include the following software packages:

- TensorFlow
- Caffe
- Theano
- Keras
- Lasagne
- Torch
- iPython/Jupyter Notebook
- Numpy
- SciPy
- Pandas
- Scikit Learn
- Matplotlib
- OpenCV

The playbook to build the DL Docker image is given below:

```

- name: Build the dl-docker image
  hosts: localhost
  gather_facts: true
  become: true
  tags: [deep-learning]

vars:
  DL_BUILD_DIR: "/tmp/dl-docker"
  DL_DOCKER_NAME: "floydhub/dl-docker"

```